

从配方到原则:统一 OPD 训练算子与受控交叉实验

Toward a Principle-Driven OPD Framework — 立项陈述与阶段性发现

2026-08-13

摘要

on-policy distillation(OPD)近年进展很快,但整体仍然是配方驱动的:一个方法往往同时改动多个训练设计——轨迹如何生成、哪些 token 被监督、用多大的词表支撑、采用哪种 KL、如何处理 teacher signal、以及最终通过什么形式形成参数更新。很多配方确实有效,但由于多个因素一起变化,我们很难回答:真正起作用的是哪一个设计选择,以及它为什么有效。

本文不再把 OPD 视为某一个固定的 loss,而是把一次完整的 OPD 更新写成一个可分解的统一训练算子,并把设计空间归纳为四个宏坐标 $\Gamma/A/C/U$;在此基础上通过受控交叉实验(controlled crossover experiments)逐坐标干预。我们在数学推理任务上、以 4B 与 1.7B 两个模型规模累计完成 30+ 组受控对比,覆盖并拆解了既有 OPD 文献中约 15 条方法线,已得到三条跨方法的结论:(i) 梯度实现 $\mathcal{R}$ 必须被视为一个独立的算子坐标——同一个名义 objective,仅改变 PG 与 direct differentiation,学习效率、种子方差与长程稳定性会同时改变;(ii) 形式迥异的设计常通过同一个底层机制影响稳定性——真正能排序稳定性的不是平均 loss 或全局梯度范数,而是极少数 token 上真正进入 optimizer 的极端更新信号;(iii) 稳定性机制 $\neq$ 学习机制——限制极端更新主要决定“什么时候坏”,并不单独决定“最终能学多好”。

最终目标不是再提出一个新的 OPD recipe,而是建立一套能够解释现有方法、指导未来设计的 principle-driven OPD framework。

目录

1 问题设定

- 1.1 现状:配方越来越多,原则依然缺失
- 1.2 一个典型的混淆:“top- k OPD”到底改了什麼
- 1.3 我们要回答的问题

2 统一 OPD 训练算子

- 2.1 算子形式
- 2.2 四个宏坐标
- 2.3 一次更新的信息流
- 2.4 为什么这个表示有用

3 实验方法:受控干预

- 3.1 只改变一个坐标
- 3.2 设计坐标 $\rightarrow$ 机制坐标

4 实验规模与覆盖

5 阶段性发现

5.1 同一个 objective, 不同 gradient realization 产生截然不同的训练行为

5.2 形式不同的设计, 可能通过同一个底层机制影响稳定性

5.3 限制极端 token-level update 能显著延缓 collapse, 但不会自动带来性能提升

6 小结

1 问题设定

1.1 现状:配方越来越多,原则依然缺失

现有 OPD 方法通常会同时修改多个训练设计维度:轨迹如何生成、哪些 token 被监督、使用多大的词表支撑、采用哪种 KL、如何处理 teacher signal,以及最终通过什么方式形成参数更新。由于这些因素经常一起变化,即使实验显示某个方法更好,也很难判断收益来自哪一处改动。

1.2 一个典型的混淆:“top- k OPD” 到底改了什么

一个看似简单的“top- k OPD”往往并不只是把监督范围从一个 sampled token 扩展到 top- k 。它通常还同时改变了

support: sampled singleton $\rightarrow$ top- k , evaluation: Monte Carlo $\rightarrow$ exact,

甚至

update realization: PG $\rightarrow$ direct.

因此,即使实验显示 top- k 方法效果更好,也很难进一步回答:收益究竟来自更丰富的 teacher 信息、更低方差的精确求值,还是 **direct gradient** 本身。类似的混淆也广泛存在于 KL 方向、trajectory filtering、token selection、signal clipping 等设计中。

1.3 我们要回答的问题

当前 OPD 面临的一个核心问题是:

我们已经积累了越来越多有效的 OPD 配方,但仍缺少可靠、可迁移的 OPD 设计原则。

我们希望把 OPD 从一组经验方法,重新表述为一个可分解的统一训练算子,并通过受控干预逐坐标回答:哪些组件真正决定 OPD 的性能、稳定性与效率?

2 统一 OPD 训练算子

2.1 算子形式

设 prompt $x \sim \mathcal{D}$,由冻结的行为策略生成 $y \sim \mu_{\bar{\theta}}(\cdot | x)$,访问状态 $s_t = (x, y_{<t})$,学生 $p_t^o = \pi_{\theta}(\cdot | s_t)$,教师 $p_t^T = \pi_T(\cdot | s_t)$ 。一次完整的 OPD 更新写作

$$g_\theta = \mathbb{E}_{\substack{x \sim \mathcal{D}, \\ y \sim \underbrace{\mu_\theta(\cdot|x)}_{\Gamma}}} \left[\underbrace{w(y) \frac{1}{\sum_t m_t} \sum_t m_t}_{\Gamma} \underbrace{\mathcal{R} \left[\underbrace{\Phi \left(\underbrace{\mathcal{Q} \left[\underbrace{D \left(\underbrace{S(N_\nu(\text{Res}_{\Omega_t} p_t^\theta)), S(N_\nu(\text{Res}_{\Omega_t} p_t^T))}_{\mathcal{A}} \right)}_{\mathcal{C}} \right)}_{\mathcal{U}} \right)}_{\mathcal{U}} \right]}_{\mathcal{U}} \right] \quad (1)$$

其中 Res_{Ω_t} 表示把分布限制到词表支撑 Ω_t 上(与求值算子 $\mathcal{Q}$ 刻意用不同记号,避免视觉混淆)。

关于两个选择权重的归一化。式 (1) 写的是分子形式;我们代码中实际交付的估计量是双重条件均值

$$\hat{g} = \frac{1}{\sum_i w_i} \sum_i w_i \left[\frac{1}{\sum_t m_{it}} \sum_t m_{it} g_{it} \right], \quad (2)$$

其估计对象是 $\mathbb{E}[\cdot | w = 1]$ 而非 $\mathbb{E}[w \cdot (\cdot)]$ 。这一区别不是形式上的:在 G 轴测得的 trajectory 保留率(0.5%–2%)下,两者相差 50–200 倍。两个选择权重(w 与 m)都被对称地归一化。¹

2.2 四个宏坐标

算子中每一块都对应 OPD 中一个非常具体的问题:

$$g_\theta = g(\theta; \Gamma, \mathcal{A}, \mathcal{C}, \mathcal{U})$$

宏坐标	组成	决定什么	典型取值
Γ	(μ, w, m)	在哪教:谁生成 rollout、哪些 trajectory 被保留、trajectory 中哪些 token 被监督	μ : on-policy / 混合 λ · w : trajectory filtering 与 reweighting · m : 位置窗口 / 随机散布 / 熵或分歧准则
$\mathcal{A}$	$(\Omega, \nu, \mathcal{S})$	老师给什么信息:只看 sampled token 还是 top- k , 是否 renormalize, 以及传递完整概率、相对 shape、rank 还是仅 candidate set	Ω : sampled singleton / top-32 / top-8 / adaptive · ν : raw / renorm / tail-bucket · $\mathcal{S}$: value / z-shape / rank / set
$\mathcal{C}$	$(D, \mathcal{Q})$	怎么比较:使用哪种 discrepancy, 以及它是在整个 support 上精确求和还是通过 sampling 估计	D : RKL / FKL / skew KL / JSD / power comparator · $\mathcal{Q}$: exact support sum / Monte-Carlo
$\mathcal{U}$	$(\Phi, \mathcal{R})$	怎么更新:teacher–student discrepancy 是否经过 clipping、compression 等 signal shaping, 以及最终通过 policy-gradient 还是 direct differentiation 更新 student	Φ : identity / soft-log / clip / positive-only clip / tanh · $\mathcal{R}$: PG / direct differentiation

2.3 一次更新的信息流

因此,不同 OPD 方法实际上都可以看成这个算子中不同位置的选择:

$$\boxed{\text{trajectory} \xrightarrow{\Gamma} \text{teacher information} \xrightarrow{\mathcal{A}} \text{discrepancy} \xrightarrow{\mathcal{C}} \text{update signal} \xrightarrow{\mathcal{U}} \Delta\theta} \quad (3)$$

2.4 为什么这个表示有用

这个表示使我们能够把过去通常绑定在一个方法名称下的多个变化拆开。例如 §1.2 的“top- k OPD”不仅改变 vocabulary support Ω ,还同时把 sampled evaluation $\mathcal{Q}$ 换成 exact evaluation、把 PG realization $\mathcal{R}$ 换成 direct differentiation。我们的目标就是通过受控交叉实验把这些原本耦合的因素逐一拆开,判断究竟哪一个真正 load-bearing 的设计选择。

3 实验方法:受控干预

3.1 只改变一个坐标

有了统一算子之后,实验目标不再是比较“哪个 OPD 方法更好”,而是做 controlled intervention:尽可能固定其余坐标,只改变算子中的一个局部选择,并通过 crossover experiment 判断这个坐标是否真正决定性能、稳定性或训练效率。

例如我们不直接比较“sampled OPD”与“top- k OPD”(后者同时改变 Ω 、 $\mathcal{Q}$ 与 $\mathcal{R}$),而是构造只改变其中一个因素的对照:

$$\text{same support} + \text{same KL} + \text{same exact evaluation}, \quad \boxed{\mathcal{R} : \text{PG} \rightarrow \text{direct}} \quad (4)$$

或固定 direct realization,仅改变支撑

$$\boxed{\Omega : \text{top-32} \rightarrow \text{top-8} \rightarrow \text{adaptive}} \quad (5)$$

以及固定支撑,仅改变尾部表示

$$\boxed{\nu : \text{renorm} \rightarrow \text{tail-bucket}} \quad (6)$$

3.2 设计坐标 $\rightarrow$ 机制坐标

这种设计使我们能够把一个复杂 OPD recipe 的收益逐层拆开,并进一步区分两类变量:

$$\boxed{\text{design coordinates} \rightarrow \text{mechanism coordinates}} \quad (7)$$

- **design coordinates** — 算法上选择了什么:support、KL、clipping、PG/direct 等;
- **mechanism coordinates** — 这些选择最终在训练中产生了什么:teacher-student support overlap、有效监督样本数、极端 token-level update signal、entropy、response length、collapse time 等。

我们目前已经发现,一个方法的名称往往并不能直接解释它为什么有效:不同的设计可以通过相同的底层机制产生相似训练行为,而同一个名义 loss 仅因为 realization 不同,也可能表现出完全不同的 dynamics。

4 实验规模与覆盖

我们首先在数学推理任务上,以 4B 与 1.7B 两个模型规模开展系统实验,累计完成 **30+** 组 **controlled comparisons**,覆盖并拆解了现有 OPD 文献中约 **15** 条 **prior-work method lines / design families**,包括 sampled RKL、top-*k* distribution matching、FKL、skew KL、JSD、adaptive support、trajectory filtering/reweighting、token-level supervision allocation、rank/set-based distillation、signal clipping/compression 等。

与逐篇复现不同,我们把这些已有方法还原到统一算子的基本坐标,并通过 crossover experiments 分离它们通常耦合在一起的设计选择。

5 阶段性发现

5.1 同一个 objective,不同 gradient realization 产生截然不同的训练行为

已有工作分别研究过 policy-gradient estimator、stop-gradient 或特定 loss 的 gradient bias,但通常把这些问题放在某个具体方法内部分析。我们的受控交叉实验进一步表明,**gradient realization** 本身应该被视为一个独立的 **OPD operator coordinate**。

在保持 teacher information、vocabulary support、normalization、discrepancy 以及 evaluation 方式全部不变时,我们只改变式 (4) 的 $\mathcal{R}$:

指标	$\mathcal{R} = \text{PG}$	$\mathcal{R} = \text{direct}$	变化
early validation (@25 步)	0.417 ± 0.094	0.638 ± 0.002	+0.221
seed variance	高	显著下降	约 47× 收窄
response length	几乎完全稳定	出现明显长序列 excursion	稳定性反转

这说明,即使两个方法优化的是同一个名义 objective,最终以什么梯度形式更新模型,仍然可以同时改变

learning efficiency, variance, long-horizon stability

因此,OPD 中“优化什么 loss”和“这个 loss 如何变成实际 update”不能被视为同一件事。很多过去看起来属于 support 或 divergence 的方法差异,实际上可能部分来自它们所采用的不同 gradient realization。

5.2 形式不同的设计,可能通过同一个底层机制影响稳定性

我们发现,表面上属于不同 operator coordinates 的设计——例如改变 discrepancy geometry、对训练信号做 clipping 或 nonlinear compression——虽然形式完全不同,却呈现出高度一致的稳定性规律。

真正能够解释这些方法差异的,并不是平均 loss、整体 gradient norm 或常见的 bulk statistics,而是极少数 token 上实际进入 optimizer 的 extreme update signal。在我们的实验中,随着该极端信号的规模逐步下降,late-stage collapse 被系统性推迟:²

极端更新信号规模	81.6	10	4.5	2.3	1
length collapse 发生步数	122	198	208	247	从未发生

相比之下,mean signal、p95 signal 与 global gradient norm 都无法给出相同的排序。这一结果提示一个重要区别:

design coordinate $\neq$ mechanism coordinate

不同的 OPD 设计可以通过完全不同的数学形式,最终改变同一个底层量;而真正控制训练 dynamics 的,可能正是这些更少数、更通用的 mechanism coordinates。因此我们的目标不仅是判断某一种 KL、clipping 或 transformation 是否有效,更希望进一步识别:哪些底层机制能够跨越不同 OPD 方法,统一解释其性能与稳定性。

5.3 限制极端 token-level update 能显著延缓 collapse,但不会自动带来性能提升

§5.2 的阶梯说明 OPD 的 late-stage stability 对极少数 token 上的 extreme update signal 非常敏感,late-stage collapse 更可能由极少量 extreme token updates 驱动,而不是整体 gradient scale。

但更重要的是:collapse 被显著推迟,并没有带来相应幅度的最终性能提升。一些训练能够稳定更久、甚至不再出现 length collapse,但最终 task performance 与更早 collapse 的配置相比并没有显著改善。因此目前结果支持

stability mechanism $\neq$ learning mechanism

也就是说,限制 extreme update 主要决定 OPD “什么时候会坏”,但并不能单独决定模型 “最终能学多好”。这意味着,单纯通过 clipping、compression 或 bounded transformation 提高训练稳定性是不够的;真正有效的 OPD 还需要同时解决 teacher information、supervision structure 与 update direction 等其他 operator coordinates。

更一般地,我们观察到:

更稳定的 OPD 并不必然是更好的 OPD

这提示 OPD 的性能与稳定性至少部分由不同的底层机制控制,而 unified operator framework 的价值之一,就是把这些机制进一步拆开。

6 小结

把一次 OPD 更新写成式 (1) 的统一算子,带来三点直接收益:

1. 可解释——既有方法不再是一串名字,而是四个宏坐标上的一组具体选择,方法之间的差异可以被定位到坐标;
2. 可干预——实验从“比较方法”变成“只动一个坐标”的受控交叉,收益归因不再依赖整包对比;
3. 可迁移——结论落在 mechanism coordinates 上,而非某个 loss 的名字,因此有机会跨方法、跨领域复用。

我们的目标不是再提出一个新的 OPD recipe,而是建立一套能够解释现有方法、指导未来设计的 **principle-driven OPD framework**。

注释

1. 本节与内部定稿 docs/UNIFIED-LOSS.md (v2 §0/§6) 一致——早期草稿只归一化了 m 而未归一化 w ,该修正已随 v2 记录在案。
2. 该阶梯上每一档对应的具体臂、逐种子曲线与判据见内部记录 docs/expansion-early-readings.md (附录 A 全量数据)。